

Real World Security

Tom Chothia, David Oswald and Ahmad Kamel

What is Cyber Security about?

- You can't fly
 - University IT can't read your signal chat messages
-

- You cannot take money from a locked iPhone
- You can't drink coffee in lecture rooms

How so we do this?

- We need to understand exactly how a system works. What rules does it follow, how are these enforced.
- We need to understand the system better than the people you made it and find where their assumptions are wrong.
- What assumption about a system are not quite right, and how can these be exploited?
- Multi layered systems make this easier. People make assumptions about the layer below.
 - Example: buffer overflow attacks,
 - Example: Speculative execution.

Ask questions!

- I'm going to move fast.
- If you are confused, then probably half the class is.
- Ask me to explain.
- Either put your hand up or just shout out.

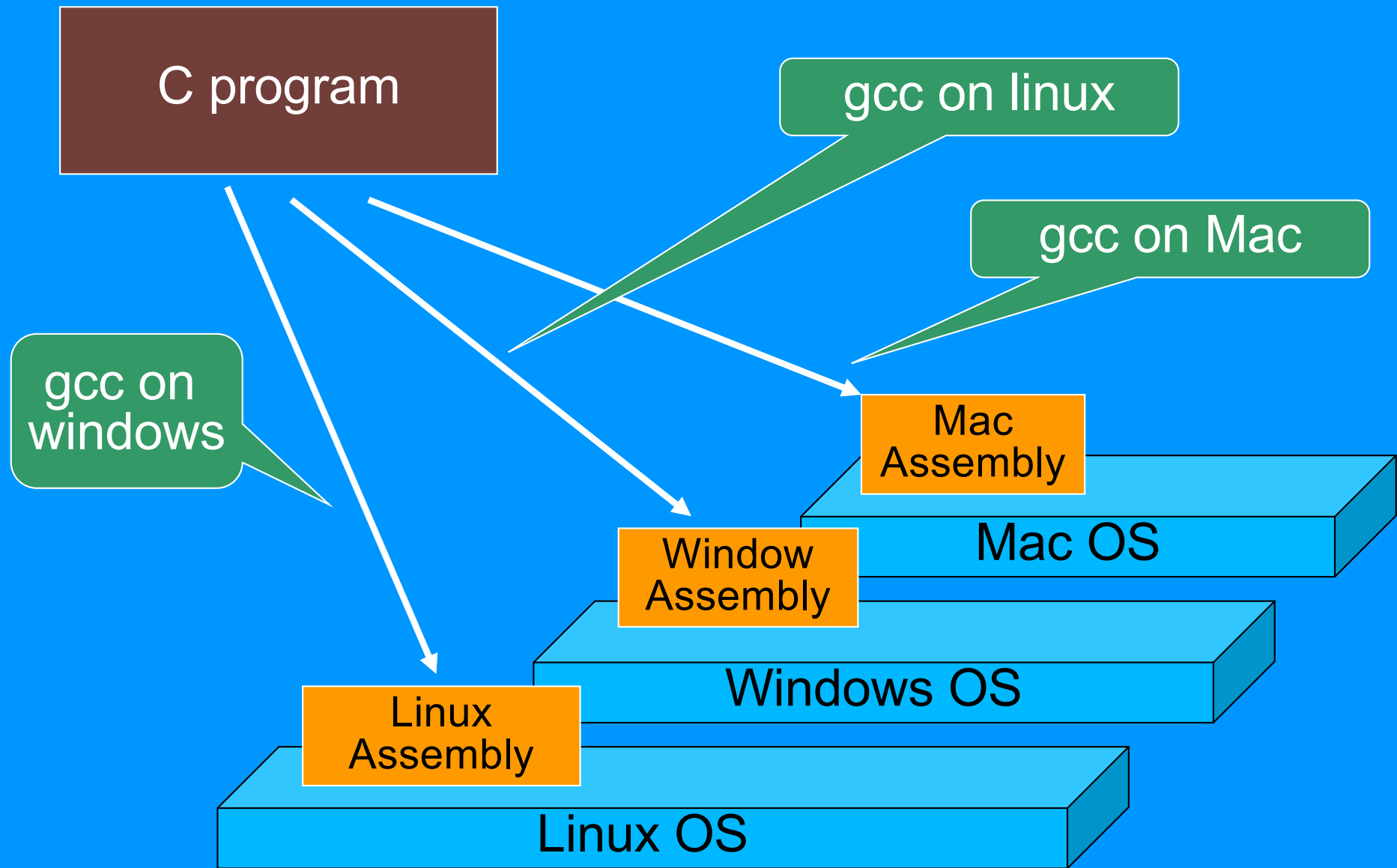


Reverse Engineering Binaries

Tom Chothia

Binaries

- Binaries are written in assembly
- Much lower level than Java byte code,
- Assembly compiled for one type of machine won't run on another.
- But the same techniques apply.



C program

X64 mac
gcc

Mac: drivers & libs

Windows: drivers & libs

Linux: drivers & libs

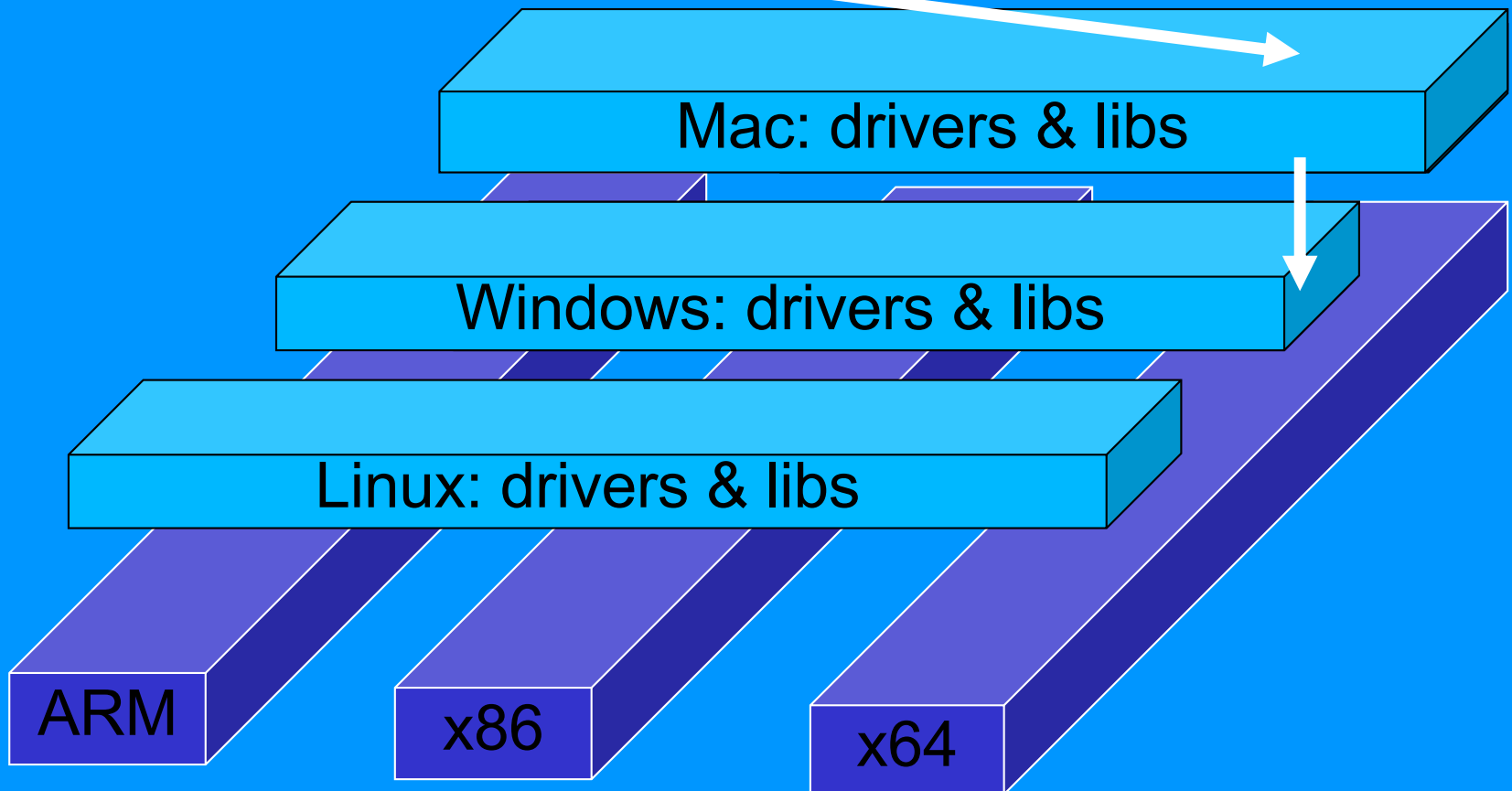
OS type:

CPU type:

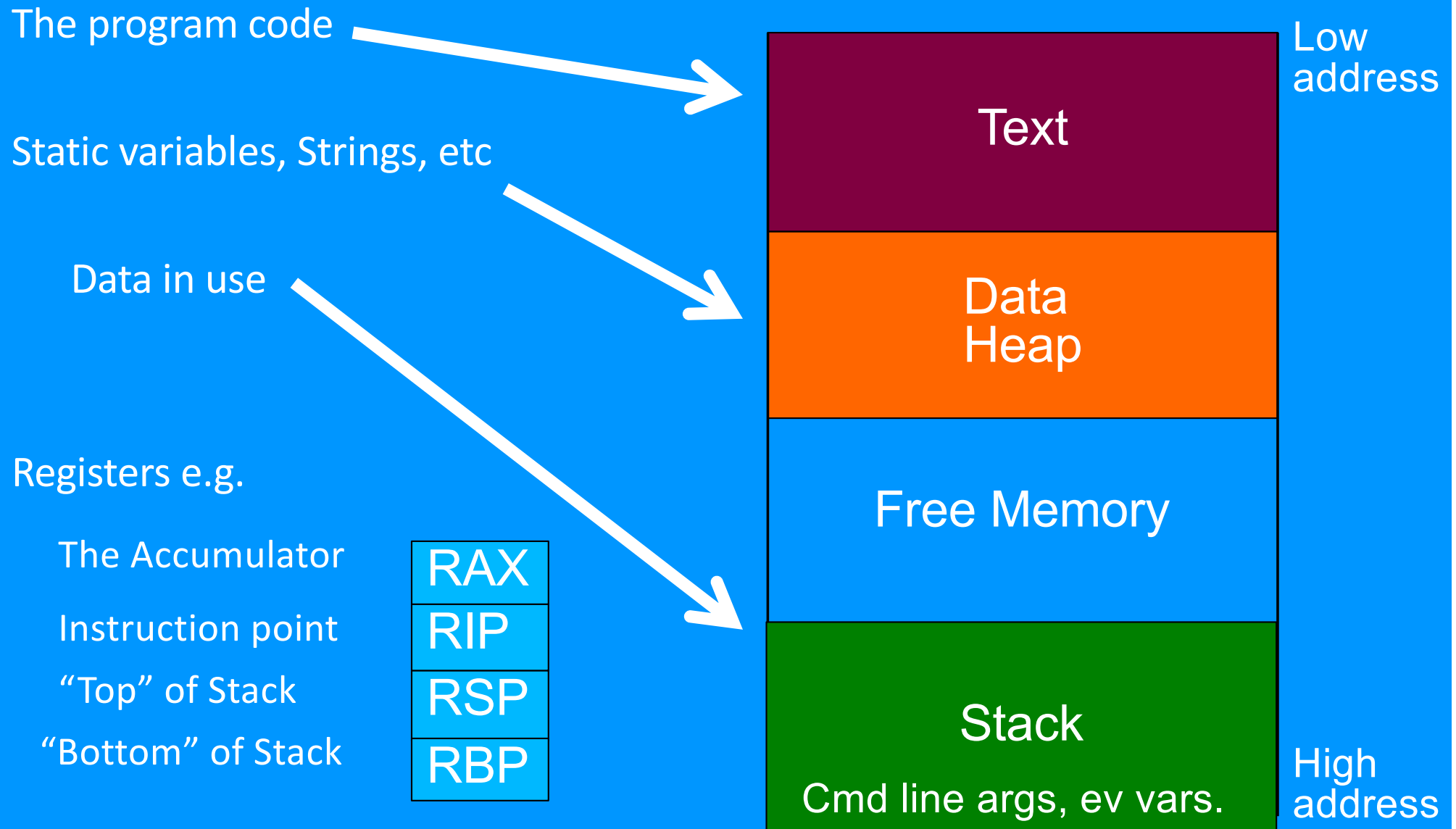
ARM

x86

x64



The x86-64 Architecture

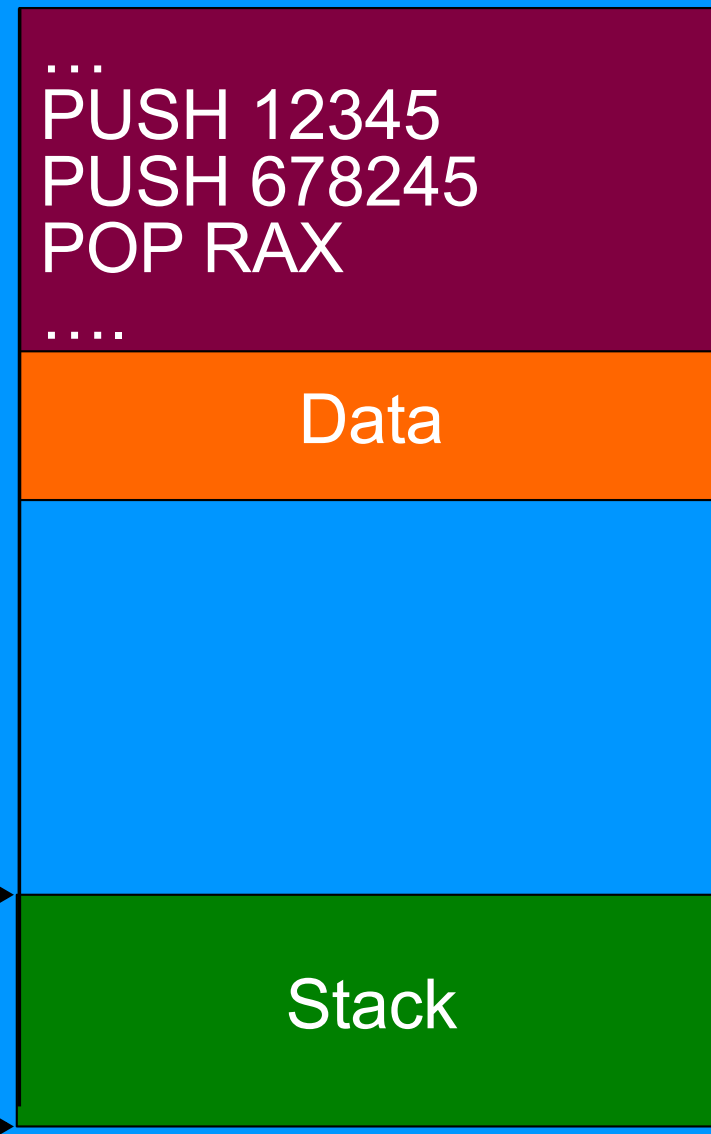


The Stack

The stack part of the memory is mostly “Last In, First Out”.

We can write and read to the top of the stack.

RAX: 0000000000000000
RIP: 000000007797F9CD
RSP: 00000000BF914EB0
RBP: 00000000BF914ED8



Sub-parts of registers can be addressed directly.

8-byte register	Bytes 0-3	Bytes 0-1	Byte 0
%rax	%eax	%ax	%al
%rcx	%ecx	%cx	%cl
%rdx	%edx	%dx	%dl
%rbx	%ebx	%bx	%bl
%rsi	%esi	%si	%sil
%rdi	%edi	%di	%dil
%rsp	%esp	%sp	%spl
%rbp	%ebp	%bp	%bpl
%r8	%r8d	%r8w	%r8b
%r9	%r9d	%r9w	%r9b
%r10	%r10d	%r10w	%r10b
%r11	%r11d	%r11w	%r11b
%r12	%r12d	%r12w	%r12b
%r13	%r13d	%r13w	%r13b
%r14	%r14d	%r14w	%r14b
%r15	%r15d	%r15w	%r15b

For floating point: XMM0, XMM1, XMM2, XMM3, XMM4, XMM5, XMM6, XMM7

32 to 64 bit: x86 to x86-64 (x64)

- The addresses are twice as long!
- Registers have a range of different names
- In x86 functions arguments are past on the stack
- In x64 the first 6 arguments are past in registers and any others on the stack.
- Main difference is that you are going to see a lot more “R”s and a far fewer “E”s.

Some x64 Commands

MOV: move value between registers

MOVxy: move value of different lengths

CALL: execute a function

RET, RETN, RETF: end a function and restart calling code.

CMP, TEST: compare 2 values

JE, JNE, JLE, ...: jump based on comparison.

LEA: like MOV but can evaluate arguments

[] : value at memory location, e.g. if rax is a memory address [rax] is the value at that address

[] : value at memory location,

- `mov rax rbx`

- Move value in rbx to rax

- `mov [rax] rbx`

- Move value in rbx to address rax points at

- `mov rax [rbx]`

- Move value rbx is pointing at to rax

- `mov [rax] [rbx]`

- CPUs are physically unable to do this!

Registers:

RAX: 600000e78000

RBX: 600000e78010

Memory:

0x600000e78000: 12345678

0x600000e78008: 11111111

0x600000e78010: 22222222

=== `mov rax [rbx]` ===>

Registers:

RAX: 22222222

RBX: 600000e78010

Memory:

0x600000e78000: 12345678

0x600000e78008: 11111111

0x600000e78010: 22222222

Calling Conventions: How functions are called

Linux

- Arguments are put in registers:
 - RDI, RSI, RDX, RCX, R8, R9, and then stack
 - XMM0, XMM1, ..., XMM7 for floating point numbers
- **Result(s) return in RAX** (and RDX) or XMM0 and XMM1.
- The function called makes new stack space by updating RSP and RBP
- The instruction pointer and old RBP are stored on the stack for when the function returns.

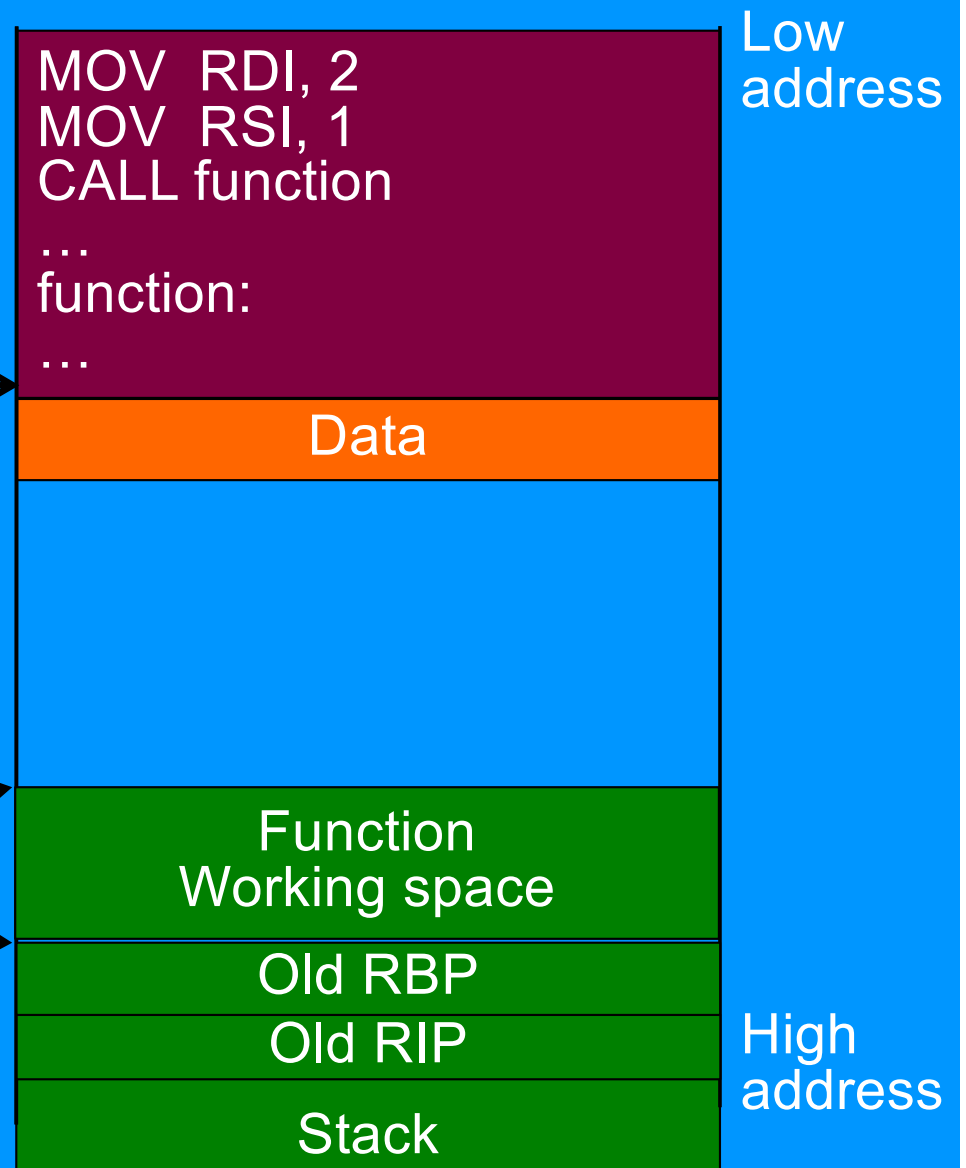
Windows is similar: arguments are in RCX, RDX, R8, R9 or XMM0, XMM1, XMM2, XMM3

32-bit (x86) similar, but all arguments passed on stack.

The Stack

```
void main () {  
    function (1,2);  
}
```

RAX: result
RIP: 000000007798D8EF
RSP: 00000000BF914C04
RBP: 00000000BF914EA0
RDI: 0000000000000002
RSI: 0000000000000001



IDA pro

- IDA pro is an Interactive DisAssembler.
- It helps a human understand binaries.
- This is the best tool for malware binary analysis, security analysis of firmware and reverse engineering.
- There are free & demo versions:
 - <http://www.hex-rays.com/>

Patching Binaries

- In IDA or Ghidra we can change the binary.
- In IDA: click on command > Edit > Patch > Change Bytes
- We can change values or commands
 - <https://defuse.ca/online-x86-assembler.htm> is useful.

cmp eax 1 = 83 7d f8 01 -> 83 7d f8 09 = cmp eax 9
add eax,1 = 83 c0 01 patched to 83 e8 01 = sub eax,1

We cannot (easily) add space! Must overwrite commands in place.

What I want you to understand ...

- ... The memory layout x86/x64
- ... the register names in x64
- ... the “calling convention”

You should be able to understand simple files in IDA and/or Ghidra

Recommendation: write some short simple C programs and look at them yourself in IDA.

Look at a binary the same as a word document, you can read it and edit it.